

整合 Android 小工具與 Google 助理

來源：<https://codelabs.developers.google.com/codelabs/appactions-widgets?hl=zh-tw>

步驟數：7

瞭解如何將 Android 小工具擴充至 Google 助理，讓 Google 助理向使用者顯示個人化小工具。

目錄

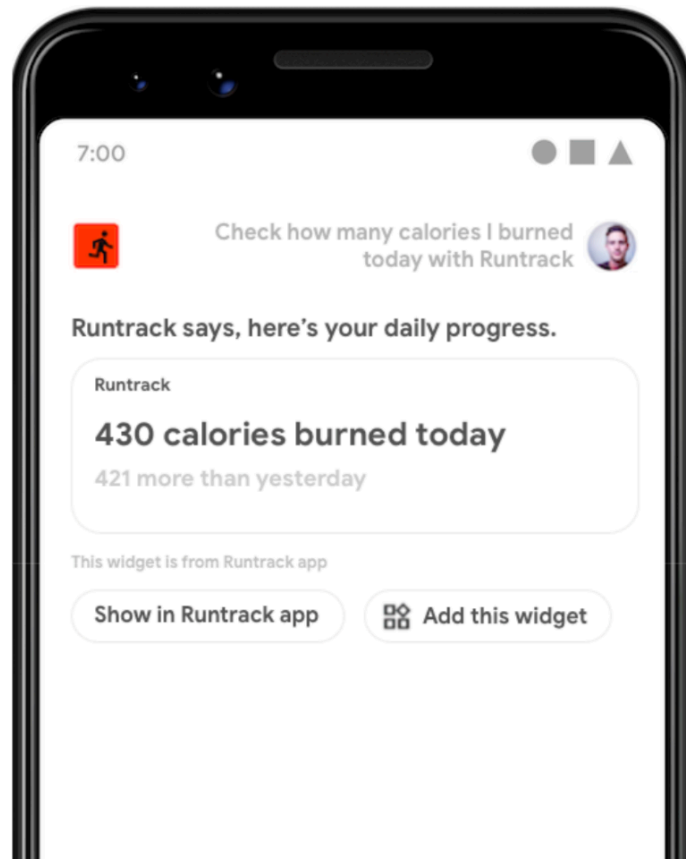
1. [1. 總覽](#)
2. [2. 瞭解運作方式](#)
3. [3. 準備開發環境](#)
4. [4. 新增應用程式動作](#)
5. [5. 啟用 Google 助理的小工具](#)
6. [6. 測試應用程式動作](#)
7. [7. 後續步驟](#)

步驟 1 · 約 3 分鐘

1. 總覽

在[第一個應用程式動作程式碼研究室](#)中，您已瞭解如何實作「健康與健身」BII 類別的[內建意圖](#) (BII)，將 Google 助理擴充至範例健身應用程式。

應用程式動作可讓使用者向 Google 助理說出「Ok Google，使用範例應用程式開始跑步」等指令，直接啟動特定應用程式功能。除了啟動應用程式，Google 助理還能向使用者顯示互動式 Android 小工具，藉此完成符合資格 BII 的要求。



注意：本程式碼研究室說明的功能目前處於 Beta 版發布階段。詳情請參閱「[應用程式動作總覽](#)」。

建構項目

在本程式碼研究室中，您將瞭解如何傳回 Android 小工具，以滿足 Google 助理使用者的要求。您也會學到如何：

- 使用 BII 參數個人化小工具。
- 在 Google 助理中為小工具提供文字轉語音 (TTS) 簡介。
- 請參閱[內建意圖參考資料](#)，判斷哪些 BII 支援小工具執行要求。

注意：「[內建意圖參考資料](#)」中符合資格的 BII 包含「Android 捷徑和 Slice」一節。

必要條件

繼續操作前，請確認開發環境已準備就緒，可供開發應用程式動作。應包含：

- 可執行 Shell 指令的終端機，並已安裝 Git。
- 最新的 [Android Studio](#) 穩定版。
- 可連上網際網路的實體或虛擬 Android 裝置。
- 登入 Android Studio、Google 應用程式和 Google 助理應用程式的 Google 帳戶。

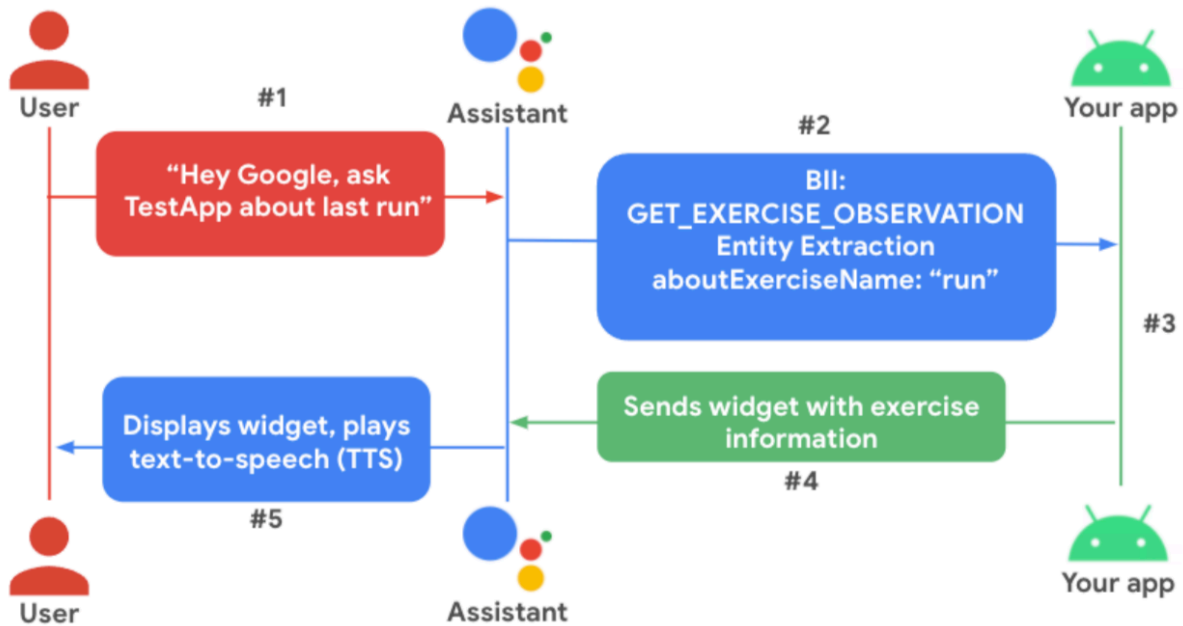
如果您使用的是實體裝置，請將裝置連線至本機開發電腦。

2. 瞭解運作方式

Google 助理會使用自然語言理解 (NLU) 技術解讀使用者要求，並比對助理內建意圖 (BII)。接著，Google 助理會將意圖對應至您在應用程式中為該意圖註冊的功能 (實作 BII)，最後，Google 助理會顯示應用程式使用功能中找到的詳細資料產生的 Android 小工具，藉此完成使用者要求。

在本程式碼研究室中，您將定義一項功能，註冊對 `GET_EXERCISE_OBSERVATION` BII 的支援。在這項功能中，您會指示 Google 助理產生 Android 意圖，傳送至 `FitActions` 小工具類別，以執行這項 BII 的要求。您會更新這個類別，為 Google 助理產生個人化小工具，並提供文字轉語音簡介，讓 Google 助理朗讀。

下圖說明這個流程：



FitActions 小工具

FitActions 範例應用程式包含運動資訊小工具，使用者可以將其新增至主畫面。這個小工具非常適合用來執行觸發 `GET_EXERCISE_OBSERVATION` BII 的使用者查詢。

小工具的運作方式

使用者將小工具新增到主畫面時，小工具會 Ping 裝置的廣播接收器。這項服務會從應用程式 `AndroidManifest.xml` 資源中的小工具接收器定義，擷取小工具的相關資訊。並使用這項資訊產生代表小工具的 `RemoteViews` 物件。

範例應用程式會定義接收器 `widgets.StatsWidgetProvider`，對應於 `StatsWidgetProvider` 類別：

```
<!-- app/src/main/AndroidManifest.xml -->

<receiver
    android:name=".widgets.StatsWidgetProvider"
    android:exported="false">
    <intent-filter>
        <action android:name="android.appwidget.action.APPWIDGET_UPDATE" />
    </intent-filter>
    <meta-data
        android:name="android.appwidget.provider"
        android:resource="@xml/stats_widget" />
</receiver>
```

`StatsWidgetProvider` 類別 (`StatsWidgetProvider.kt`) 會管理 `StatsWidget` 物件建立流程。負責處理下列事項：

- 建立小工具例項，並填入應用程式資料庫中的運動資料。
- 使用 `formatDataAndSetWidget()` 格式化運動資料，方便閱讀。
- 如果沒有運動資料，請使用 `setNoActivityDataWidget()` 提供預設值。

新增 Google 助理支援

在本程式碼研究室中，您將更新範例應用程式，以處理應用程式動作功能。包括：

1. 將 `GET_EXERCISE_OBSERVATION` BII 功能設定為傳回 `StatsWidget` 物件的執行個體。
 2. 更新 `StatsWidget` 類別，使用應用程式動作功能，例如：
 - 使用 BII 參數，讓使用者可以問「Ok Google，在範例應用程式上顯示我的跑步統計資料」等問題，查看特定運動的統計資料。
 - 提供文字轉語音簡介字串。
 - 管理特殊情況，例如使用者查詢未包含運動類型參數。
-

步驟 3 · 約 15 分鐘

3. 準備開發環境

下載基礎檔案

執行下列指令，複製範例應用程式的 [GitHub 存放區](#)：

```
git clone --branch start-widget-codelab https://github.com/actions-on-google/appactions-fitness-kotlin.git
```

複製存放區後，請按照下列步驟在 Android Studio 中開啟：

1. 在「Welcome to **Android Studio**」對話方塊中，按一下「Import project」。
2. 找出並選取複製存放區的資料夾。

如要查看代表完成程式碼研究室的應用程式版本，請使用 `--branch master` 旗標複製範例應用程式存放區。

更新 Android 應用程式 ID

更新應用程式的應用程式 ID，即可在測試裝置上清楚識別應用程式，並避免在將應用程式上傳至 Play 管理中心時發生「套件名稱重複」錯誤。如要更新應用程式 ID，請開啟 `app/build.gradle`：

```
android {  
    ...  
    defaultConfig {  
        applicationId "com.MYUNIQUENAME.android.fitactions"  
        ...  
    }  
}
```

將 `applicationId` 欄位中的「MYUNIQUENAME」替換為專屬名稱。

注意：更新 `applicationId` 後，Android Studio 會顯示「Gradle files have changed since last project sync...」警告。按一下「立即同步」，避免上傳應用程式時發生問題。

安裝測試外掛程式

[Google 助理外掛程式](#)可讓您在測試裝置上測試應用程式動作。這項功能會透過 Android 裝置上的 Google 應用程式，將資訊傳送給 Google 助理。如果您還沒有外掛程式，請按照下列步驟安裝：

1. 依序前往「File」>「Settings」（MacOS 中則為「Android Studio」>「Preferences」）。
2. 在「Plugins」部分，前往「Marketplace」**Marketplace**並搜尋「Google Assistant」。您也可以手動[下載](#)及安裝測試工具。
3. 安裝工具，然後重新啟動 Android Studio。

在裝置上測試應用程式

在對應用程式進行更多變更之前，請先瞭解範例應用程式的功能。

在測試裝置上執行應用程式：

1. 在 Android Studio 中選取實體或虛擬裝置，然後依序選取「Run」>「Run app」，或按一下工具列中的「Run」圖示



。

2. 長按主畫面按鈕設定 Google 助理，並確認是否正常運作。如果尚未登入，請先在裝置上登入 Google 助理。

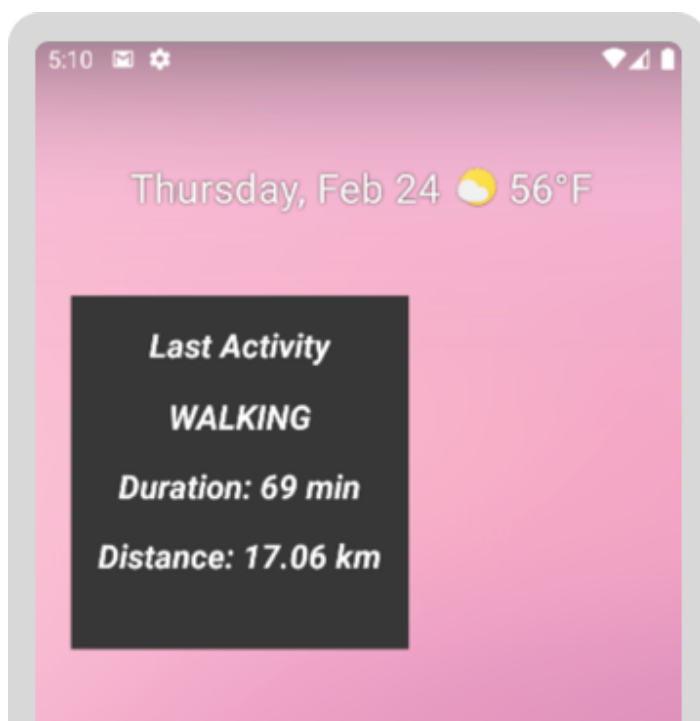
注意：建議使用 Android 10 (API 級別 30) 以上版本，不過應用程式動作最多可以支援到 [Android 5 \(API 級別 21\)](#) 的裝置。

如要進一步瞭解 Android 虛擬裝置，請參閱「[建立及管理虛擬裝置](#)」。

簡單瀏覽應用程式，瞭解其功能。應用程式會預先填入 10 項運動活動，並在第一個檢視畫面中顯示這項資訊。

試用現有小工具

1. 輕觸「主畫面」按鈕，前往測試裝置的主畫面。
2. 長按主畫面上的空白處，然後選取「小工具」。
3. 在小工具清單中向下捲動至「FitActions」**FitActions**。
4. 長按 FitActions 圖示，然後將小工具放在主畫面上。



4. 新增應用程式動作

在這個步驟中，您會新增 `GET_EXERCISE_OBSERVATION` BII 功能。方法是在 `shortcuts.xml` 中新增 `capability` 元素。這項功能會指定功能的觸發方式、BII 參數的使用方式，以及要叫用哪些 Android Intent 來滿足要求。

1. 將新的 `capability` 元素新增至範例專案的 `shortcuts.xml` 資源，並採用下列設定：

```
<!-- fitnessactions/app/src/main/res/xml/shortcuts.xml -->

<capability android:name="actions.intent.GET_EXERCISE_OBSERVATION">
  <app-widget
    android:identifier="GET_EXERCISE_OBSERVATION"
    android:targetClass="com.devrel.android.fitactions.widgets.StatsWidgetProvider"
    android:targetPackage="PUT_YOUR_APPLICATION_ID_HERE">
    <parameter
      android:name="exerciseObservation.aboutExercise.name"
      android:key="aboutExerciseName"
      android:required="true">
    </parameter>
    <extra android:name="hasTts" android:value="true"/>
  </app-widget>
  <!-- Add Fallback Intent-->
</capability>
```

將 `android:targetPackage` 值 `PUT_YOUR_APPLICATION_ID_HERE` 替換為您的專屬 `applicationId`。

這項功能會將 `GET_EXERCISE_OBSERVATION` BII 對應至 `app-widget` 意圖，因此當 BII 觸發時，小工具會例項化並顯示給使用者。

在觸發小工具前，Google 助理會從使用者查詢中擷取支援的 BII 參數。本程式碼研究室需要 BII 參數

`exerciseObservation.aboutExercise.name`，代表使用者要求的運動類型。應用程式支援三種運動類型：「跑步」、「步行」和「騎自行車」。您可提供[內嵌目錄](#)，將這些支援的值告知 Google 助理。

2. 在 `shortcuts.xml` 中，於 `GET_EXERCISE_OBSERVATION` 功能上方新增這項設定，定義這些目錄元素：

```

<!-- shortcuts.xml -->

<!-- shortcuts are bound to the GET_EXERCISE_OBSERVATION capability and
represent the types of exercises supported by the app. -->

<shortcut
  android:shortcutId="running"
  android:shortcutShortLabel="@string/activity_running">
  <capability-binding android:key="actions.intent.GET_EXERCISE_OBSERVATION">
    <parameter-binding
      android:key="exerciseObservation.aboutExercise.name"
      android:value="@array/runningSynonyms" />
    </capability-binding>
  </shortcut>

<shortcut
  android:shortcutId="walking"
  android:shortcutShortLabel="@string/activity_walking">
  <capability-binding android:key="actions.intent.GET_EXERCISE_OBSERVATION">
    <parameter-binding
      android:key="exerciseObservation.aboutExercise.name"
      android:value="@array/walkingSynonyms" />
    </capability-binding>
  </shortcut>

<shortcut
  android:shortcutId="cycling"
  android:shortcutShortLabel="@string/activity_cycling">
  <capability-binding android:key="actions.intent.GET_EXERCISE_OBSERVATION">
    <parameter-binding
      android:key="exerciseObservation.aboutExercise.name"
      android:value="@array/cyclingSynonyms" />
    </capability-binding>
  </shortcut>

<capability android:name="actions.intent.GET_EXERCISE_OBSERVATION">
  <!-- ... -->
</capability>

```

新增備用意圖

如果使用者查詢缺少功能所需的參數，導致系統無法執行查詢，就會觸發備用意圖。 [GET_EXERCISE_OBSERVATION](#) 功能需要 `exerciseObservation.aboutExercise.name` 參數，該參數由 `android:required="true"` 屬性指定。在這些情況下，即使查詢中未提供任何參數，Google 助理仍會要求您定義備用意圖，確保要求順利執行。

1. 在 `shortcuts.xml` 中，使用下列設定將備用意圖新增至 [GET_EXERCISE_OBSERVATION](#) 功能：

```
<!-- shortcuts.xml -->

<capability android:name="actions.intent.GET_EXERCISE_OBSERVATION">

  <app-widget>
    <!-- ... -->
  </app-widget>

  <!-- Fallback intent with no parameters needed to successfully execute.-->
  <intent
    android:identifier="GET_EXERCISE_OBSERVATION_FALLBACK"
    android:action="android.intent.action.VIEW"
    android:targetClass="com.devrel.android.fitactions.widgets.StatsWidgetProvider">
  </intent>
</capability>
```

在這個範例設定中，備用執行要求是 Android 意圖，其 `Extra` 資料中沒有任何參數。

注意：在本範例中，備用 `intent` 元素列在 `app-widget` 標記之後，因為這是 BII 的全方位執行要求，不需要參數即可成功。舉例來說，如果使用者問「Ok Google，在範例應用程式上顯示我的統計資料。」，就會觸發這項功能。

5. 啟用 Google 助理的小工具

建立 `GET_EXERCISE_OBSERVATION` 功能後，請更新小工具類別，支援應用程式動作語音呼叫。

新增小工具擴充功能程式庫

「應用程式動作」[小工具擴充功能程式庫](#)可以改善小工具功能，提供以語音為主的 Google 助理體驗。具體來說，這項功能可讓您為小工具提供自訂的文字轉語音簡介。

1. 將小工具擴充功能程式庫依附元件新增至範例應用程式 `/app/build.gradle` 資源：

```
// app/build.gradle

dependencies {
    //...
    implementation "com.google.assistant.appactions:widgets:0.0.1"
}
```

在 Android Studio 顯示的警告方塊中，按一下「Sync Now」。每次 `build.gradle` 變更後都進行同步，有助於避免建構應用程式時發生錯誤。

新增小工具服務

[服務](#)是一種應用程式元件，可在背景執行長時間作業。應用程式必須提供服務來處理小工具要求。

1. 使用下列設定，將服務新增至範例應用程式的 `AndroidManifest.xml` 資源：

```
<!-- AndroidManifest.xml -->
<service
    android:name=".widgets.StatsWidgetProvider"
    android:enabled="true"
    android:exported="true">
    <intent-filter>
        <action
            android:name="com.google.assistant.appactions.widgets.PIN_APP_WIDGET" />
    </intent-filter>
</service>
```

在觸發小工具執行要求的語音查詢期間，Google 助理會使用這項服務將要求傳送至應用程式。這項服務會收到要求和 BII 資料。這項服務會使用這項資料產生 `RemoteView` 小工具物件，以便在 Google 助理中轉譯。

更新小工具類別

您的應用程式現在已設定為將 `GET_EXERCISE_OBSERVATION` 功能要求傳送至小工具類別。接著，更新 `StatsWidget.kt` 類別，使用 BII 參數值產生專為使用者要求打造的微件例項。

1. 開啟 `StatsWidget.kt` 類別，然後匯入應用程式動作小工具擴充功能程式庫：

```
// StatsWidget.kt

// ... Other import statements
import com.google.assistant.appactions.widgets.AppActionsWidgetExtension
```

2. 新增下列私有變數，用於判斷應在小工具中填入的資訊：

```
// StatsWidget.kt

private val hasBii: Boolean
private val isFallbackIntent: Boolean
private val aboutExerciseName: String
private val exerciseType: FitActivity.Type
```

3. 新增 `init` 函式，讓類別使用從 Google 助理傳遞的小工具選項資料：

```
// StatsWidget.kt

init {
    val optionsBundle = appWidgetManager.getAppWidgetOptions(appWidgetId)
    val bii = optionsBundle.getString(AppActionsWidgetExtension.EXTRA_APP_ACTIONS_BII)
    hasBii = !bii.isNullOrEmpty()
    val params =
optionsBundle.getBundle(AppActionsWidgetExtension.EXTRA_APP_ACTIONS_PARAMS)

    if (params != null) {
        isFallbackIntent = params.isEmpty
        if (isFallbackIntent) {
            aboutExerciseName = context.resources.getString(R.string.activity_unknown)
        } else {
            aboutExerciseName = params.get("aboutExerciseName") as String
        }
    } else {
        isFallbackIntent = false
        aboutExerciseName = context.resources.getString(R.string.activity_unknown)
    }
    exerciseType = FitActivity.Type.find(aboutExerciseName)
}
```

接下來，我們將逐步說明這些更新如何讓 `StatsWidget.kt` 類別回應 `GET_EXERCISE_OBSERVATION` 功能產生的 Android 意圖：

- `optionsBundle` = `Bundle`
 - 套件是物件，可用於程序界限之間、具有意圖的活動之間，以及儲存設定變更的暫時狀態。Google 助理會使用 `Bundle` 物件將設定資料傳遞至小工具。
- `bii` = `actions.intent.GET_EXERCISE_OBSERVATION`

- 您可以使用 `AppActionsWidgetExtension` 從套件取得 BII 名稱。
- `hasBii = true`
 - 檢查是否有 BII。
- `params = Bundle[{aboutExerciseName=running}]`
 - 應用程式動作產生的特殊 Bundle 會巢狀內嵌在小工具選項 Bundle 中。其中包含 BII 的鍵/值組合。在本例中，值 `running` 是從範例查詢「Ok Google，在範例應用程式上顯示我的跑步統計資料」中擷取。
- `isFallbackIntent = false`
 - 檢查意圖 Extras 中是否有必要的 BII 參數。
- `aboutExerciseName = running`
 - 取得 `aboutExerciseName` 的意圖 Extras 值。
- `exerciseType = RUNNING`
 - 使用 `aboutExerciseName` 查詢對應的資料庫類型物件。

`StatsWidget` 類別現在可以處理傳入的應用程式動作 Android intent 資料，請更新小工具建立流程邏輯，檢查小工具是否由應用程式動作觸發。

4. 在 `StatsWidget.kt` 中，將 `updateAppWidget()` 函式替換為下列程式碼：

```
// StatsWidget.kt

fun updateAppWidget() {
    /**
     * Checks for App Actions BII invocation and if BII parameter data is present.
     * If parameter data is missing, use data from last exercise recorded to the
     * fitness tracking database.
     */
    if (hasBii && !isFallbackIntent) {
        observeAndUpdateRequestedExercise()
    } else observeAndUpdateLastExercise()
}
```

先前的程式碼會參照新函式 `observeAndUpdateRequestedExercise`。這個函式會使用 Android 意圖傳遞的 `exerciseType` 參數資料，產生小工具資料。

5. 使用下列程式碼新增 `observeAndUpdateRequestedExercise` 函式：

```
// StatsWidget.kt

/**
 * Create and observe the last exerciseType activity LiveData.
 */
private fun observeAndUpdateRequestedExercise() {
    val activityData = repository.getLastActivities(1, exerciseType)

    activityData.observeOnce { activitiesStat ->
        if (activitiesStat.isNotEmpty()) {
            formatDataAndSetWidget(activitiesStat[0])
            updateWidget()
        } else {
            setNoActivityDataWidget()
            updateWidget()
        }
    }
}
```

在上述程式碼中，使用應用程式中的現有**存放區**類別，從應用程式的本機資料庫擷取健身資料。這個類別提供的 API 可簡化資料庫存取作業。存放區會在對資料庫執行查詢時，公開 **LiveData** 物件。您會在程式碼中觀察此 **LiveData**，以便擷取最新的健身活動。

注意：LiveData 是可觀察的資料容器類別。和一般可觀測物件不同，LiveData 具備生命週期感知功能，代表會遵循其他應用程式元件 (例如活動、片段或服務) 的生命週期。此感知功能可確保 LiveData 只會更新處於活動生命週期狀態的應用程式元件觀測器。在本程式碼研究室中，我們使用的是單一用途的觀察器。

啟用文字轉語音功能

您可以在顯示小工具時，讓 Google 助理朗讀一段文字轉語音 (TTS) 字串。建議加入這項屬性，為小工具提供語音內容。這項功能是由應用程式動作小工具擴充功能程式庫提供，可讓您設定在 Google 助理中與小工具一起顯示的文字和文字轉語音簡介。

提供 TTS 簡介的合適位置是 `formatDataAndSetWidget` 函式，這個函式會格式化從應用程式資料庫傳回的活動資料。

1. 在 `StatsWidget.kt` 中，將下列程式碼新增至 `formatDataAndSetWidget` 函式：

```
// StatsWidget.kt

private fun formatDataAndSetWidget(
    activityStat: FitActivity,
) {
    // ...

    // Add conditional for hasBii for widget with data
    if (hasBii) {
        // Formats TTS speech and display text for Assistant
        val speechText = context.getString(
            R.string.widget_activity_speech,
            activityExerciseTypeFormatted,
            formattedDate,
            durationInMin,
            distanceInKm
        )
        val displayText = context.getString(
            R.string.widget_activity_text,
            activityExerciseTypeFormatted,
            formattedDate
        )
        setTts(speechText, displayText)
    }
}
```

上述程式碼會參照兩個字串資源：一個用於語音，另一個用於文字。如需 TTS 建議，請觀看小工具影片的「[文字轉語音樣式建議](#)」部分。範例也會參照 `setTts`，這個新函式會將 TTS 資訊提供給小工具例項。

2. 使用下列程式碼，將這個新的 `setTts` 函式新增至 `StatsWidget.kt`：

```
// StatsWidget.kt

/**
 * Sets TTS to widget
 */
private fun setTts(
    speechText: String,
    displayText: String,
) {
    val appActionsWidgetExtension: AppActionsWidgetExtension =
        AppActionsWidgetExtension.newBuilder(appWidgetManager)
            .setResponseSpeech(speechText) // TTS to be played back to the user
            .setResponseText(displayText) // Response text to be displayed in Assistant
            .build()

    // Update widget with TTS
    appActionsWidgetExtension.updateWidget(appWidgetId)
}
```


最後，當運動資料庫針對要求的運動類型傳回空白資料時，請設定 TTS 資訊，完成 TTS 邏輯。

3. 使用下列程式碼更新 `StatsWidget.kt` 中的 `setNoActivityDataWidget()` 函式：

```
// StatsWidget.kt

private fun setNoActivityDataWidget() {
    // ...
    // Add conditional for hasBii for widget without data
    if (hasBii) {
        // formats speech and display text for Assistant
        // https://developers.google.com/assistant/app/widgets#library
        val speechText =
            context.getString(R.string.widget_no_activity_speech, aboutExerciseName)
        val displayText =
            context.getString(R.string.widget_no_activity_text)

        setTts(speechText, displayText)
    }
}
```

6. 測試應用程式動作

在開發期間，請使用 [Google 助理外掛程式](#)，在測試裝置上預覽 Google 助理應用程式動作。您可以使用這項工具調整應用程式動作的意圖參數，測試動作如何處理使用者要求 Google 助理執行動作的各種方式。

製作試聽內容

如要使用外掛程式測試應用程式動作，請按照下列步驟操作：

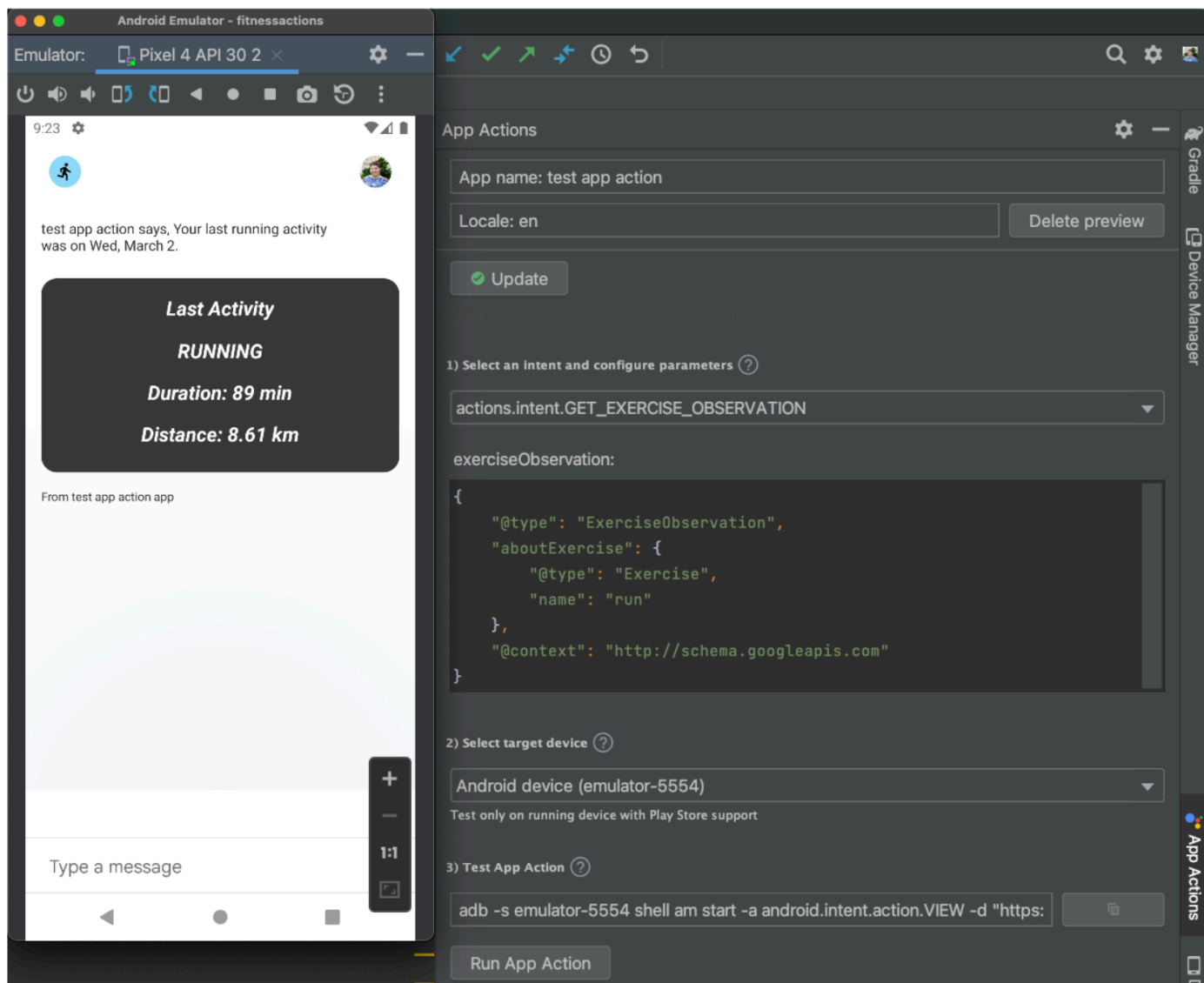
1. 依序前往「Tools」(工具) > 「Google Assistant」(Google 助理) > 「App Actions Test Tool」(應用程式動作測試工具)。
系統可能會要求您使用 Google 帳戶[登入](#) Android Studio。
2. 按一下「Create Preview」。如果系統要求，請詳閱並接受應用程式動作政策和服務條款。

測試預期的運動類型

在測試工具中按照下列步驟操作，傳回顯示應用程式中最後一次執行完成相關資訊的小工具：

1. 在工具要求選取及設定 BII 的第一個步驟中，選取 `actions.intent.GET_EXERCISE_OBSERVATION`。
2. 在「exerciseObservation」**exerciseObservation**方塊中，將預設的運動名稱從 `climbing` 更新為 `run`。
3. 按一下「執行應用程式動作」。

提示：如果執行測試工具時遇到問題，請參閱[疑難排解指南](#)。

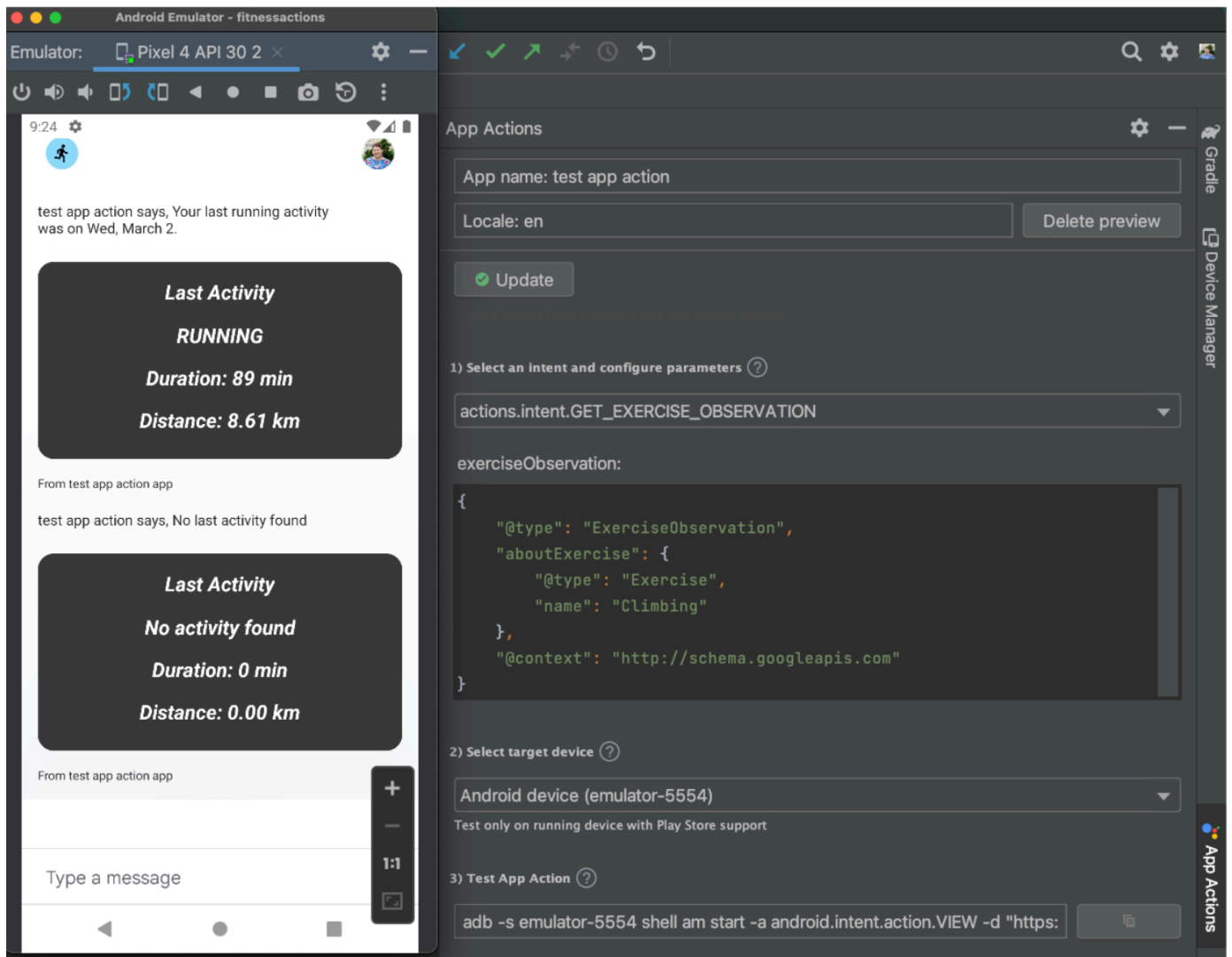


測試非預期的運動類型

如要在測試工具中測試非預期的運動類型，請按照下列步驟操作：

1. 在 **exerciseObservation** 方塊中，將 `name` 值從 `Run` 更新為 `Climbing`。
2. 按一下「執行應用程式動作」。

Google 助理應回傳顯示「找不到活動」資訊的小工具。



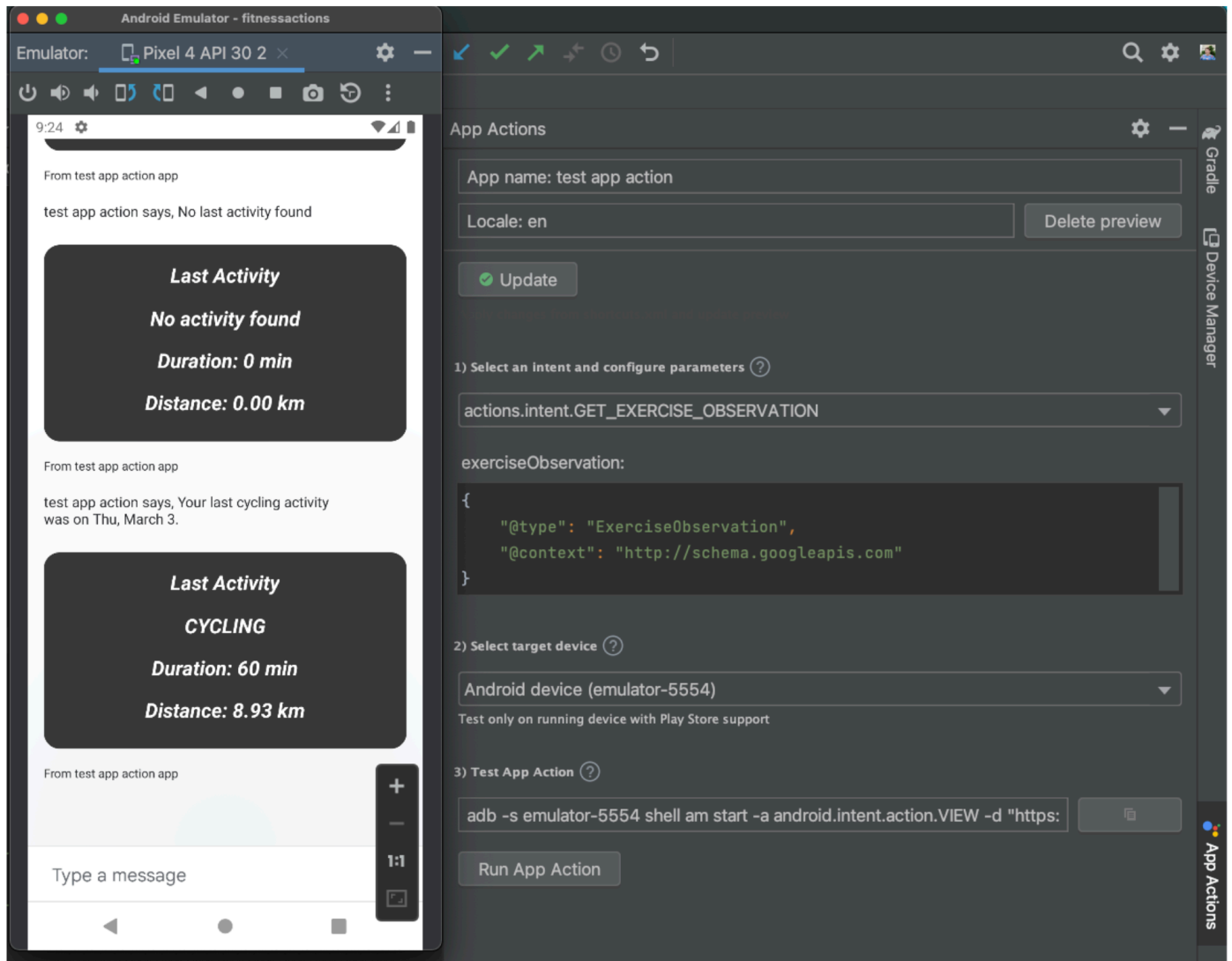
測試備用意圖

觸發後備意圖的查詢應傳回小工具，顯示任何運動類型的最後記錄活動資訊。

如要測試備用意圖，請按照下列步驟操作：

1. 在 **exerciseObservation** 方塊中，刪除 `aboutExercise` 物件。
2. 按一下「執行應用程式動作」。

Google 助理應傳回顯示上次完成運動資訊的小工具。



步驟 7 · 約 0 分鐘

7. 後續步驟

恭喜！

現在，您可以使用 Android 小工具搭配 Google 助理，完成使用者的查詢。

涵蓋內容

在本程式碼研究室中，您已瞭解如何：

- 在 BII 中新增應用程式小工具。
- 修改小工具，從 Android Extras 存取參數。

後續步驟

現在，您可以嘗試進一步調整健身應用程式。如要參考完成的專案，請參閱 GitHub 上的[主要存放區](#)。

以下提供一些建議，協助您進一步瞭解如何使用應用程式動作擴充這個應用程式：

- 如要進一步瞭解如何將應用程式擴充至 Google 助理，請參閱應用程式動作[內建意圖參考資料](#)。

如要繼續使用 Google 助理開發動作，請參閱下列資源：

- developers.google.com/assistant/app：Google 助理應用程式動作的官方說明文件網站。
- [應用程式動作範例索引](#)：範例應用程式和程式碼，可探索應用程式動作功能。
- [Actions on Google GitHub 存放區](#)：範例程式碼和程式庫。
- [r/GoogleAssistantDev](#)：專為使用 Google 助理開設的官方 Reddit 社群。

歡迎在 Twitter 追蹤 [@ActionsOnGoogle](#)，隨時掌握最新公告，並在推文中加上 [#appactions](#)，分享您開發的內容！

意見調查

最後，請[填寫這份問卷](#)，提供您對本程式碼研究室的意見。
